Task API: Docker + Postgres Migration

This document explains what was done to move the Task API from in-memory / SQLite to Postgres running in Docker, what each file does, and - with special focus - what happens when you stop Docker and the database.

Goal: Run Postgres in Docker with a volume so data persists, connect the service to it, and start app + database together with one command: docker compose up.

1. What happened - before vs after

Before

- Single file main.py with SQLite code baked into each route (get_db, direct SQL).
- DB path hard-coded to tasks.db, no .env, no Docker, no repository layer.
- Data handling mixed with HTTP handling in every route.

After

- New repositories.py holds all data access behind a TaskRepository interface.
- PostgresTaskRepository implements the same interface using psycopg2.
- main.py only calls repository.list_tasks, get_task, create_task, etc. Routes and URLs unchanged.
- Connection string comes from .env. Postgres + app start with docker compose up.
- Named volume pg_data keeps data alive across restarts.

Result: switching storage changes only one file (repositories.py). Service and routes prove the layering by staying unchanged.

2. What each file does

repositories.py - NEW - the most important file

Defines the contract and the Postgres implementation. This is the file that makes the swap possible.

```
class TaskRepository:
  list_tasks() -> list
  get_task(id) -> dict or None
  create_task(title) -> dict
  update_task(id, title, done) -> dict
  delete_task(id) -> None

class PostgresTaskRepository(TaskRepository):
  uses psycopg2 + RealDictCursor
  %s placeholders, RETURNING * on insert
```

- list_tasks: SELECT * FROM tasks ORDER BY id
- get_task: SELECT * WHERE id = %s, returns None if missing (route turns it into 404)
- create_task: INSERT ... RETURNING *, commits, returns new row
- update_task: reads current row, fills None fields, UPDATE, returns updated row
- delete_task: checks existence, DELETE, raises ValueError if not found (route turns into 404)

main.py - MODIFIED - routes unchanged

Before: each route opened sqlite3, ran SQL, closed connection. After: each route calls the repository.

```
repository = PostgresTaskRepository(CONNECTION_STRING)

@app.get('/tasks')
def get_tasks_db():
  return repository.list_tasks()

@app.post('/tasks', status_code=201)
def create_task(body: TaskCreate):
  return repository.create_task(body.title)
```

Decorators, paths, status codes, and JSON shapes are identical. Only the inside changed from SQL to repository calls. That is the architecture proving itself.

init_db.sql - NEW - one SQL file for the table

```
CREATE TABLE IF NOT EXISTS tasks (
  id SERIAL PRIMARY KEY,
  title TEXT NOT NULL,
  done BOOLEAN DEFAULT FALSE
);
```

Plus a DO block that seeds 3 rows (Learn SQL, Build CRUD API, Connect database) only if the table is empty. Single source of truth for schema.

docker-compose.yml - NEW - app + database together

```
services:
  db:
    image: postgres:16-alpine
    volumes:
      - pg_data:/var/lib/postgresql/data
    ports: ["5432:5432"]
  app:
    build: .
    depends_on: db (healthy)
    ports: ["8000:8000"]
volumes:
  pg_data:
```

- db: Postgres 16, env POSTGRES_USER/PASSWORD/DB, volume pg_data for persistence.
- app: built from Dockerfile, waits for db healthy, gets POSTGRES_URL, serves on 8000.
- One command runs all: docker compose up --build.

Dockerfile - NEW

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Installs deps first for layer caching, then copies code, then starts uvicorn.

requirements.txt - NEW

```
fastapi
uvicorn
psycopg2-binary
python-dotenv
```

psycopg2-binary talks to Postgres. python-dotenv loads .env locally.

.env and .env.example - NEW

```
POSTGRES_URL=postgresql://postgres:postgres@db:5432/tasks
```

.env is gitignored and holds the real secret. .env.example is committed so others know the format. Inside compose, host is db. Locally without compose, use localhost.

README.md - MODIFIED

Now documents Docker quickstart, architecture (only repositories.py changes), and persistence proof steps for restart test.

3. FOCUS: stopping Docker and the database

This is the core of the task: what survives, what is deleted, and why.

Where is the data?

Postgres stores its files at /var/lib/postgresql/data inside the db container. The compose file mounts named volume pg_data there. That means writes go to a Docker-managed volume on your host disk, not to the container writable layer. Deleting the container does NOT delete the volume.

Stop cases

- Ctrl+C (foreground stop): containers stop but are not removed. Volumes stay. Next up reuses same data.
- docker compose stop: stops containers, keeps containers + volumes. Next up reuses same data.
- docker compose down: stops AND removes containers + network, but KEEPS volumes by default. Next up reattaches pg_data, data still there. This is the normal restart test.
- docker compose down -v: removes containers AND volumes. pg_data deleted. ALL DATA LOST. Use only when you want a clean slate.
- docker compose up --build: rebuilds app image, reuses existing pg_data unless you passed -v before.

Why this matters

Without a volume, every docker compose down would wipe the database. With pg_data, you can restart app and container and rows are still there. That is the moment the project stops being a demo.

```
docker volume ls  # see pg_data
# create -> down -> up -> data still present = persistence proven
```

4. Persistence proof - how to check

Follow these exact steps and record output in README:

- Step 1: docker compose up --build, open http://localhost:8000/docs
- Step 2: POST /tasks with title Task that survives restart, note returned id.
- Step 3: GET /tasks confirms the row exists.
- Step 4: docker compose down (no -v).
- Step 5: docker compose up --build again.
- Step 6: GET /tasks/{id} - same row returned. If yes, persistence is proven.

If you run with -v in step 4, the row will be gone. That is expected and confirms the volume is what protects data.

5. Project structure

```
learn_backend/
  main.py           # routes, unchanged API, uses repository
  repositories.py   # interface + Postgres implementation
  init_db.sql       # table + seed
  Dockerfile        # app image
  docker-compose.yml# db + app + pg_data volume
  .env              # real connection (gitignored)
  .env.example      # committed example
  requirements.txt  # deps
  README.md         # docs + proof
```

6. Conclusion

Docker gives you a real Postgres. SQL file gives you a repeatable schema. .env gives you config without secrets in git. Repository layer gives you swap-in-one-file. Volume gives you survival across restarts. Together, docker compose up runs the whole stack, and every later week (jobs, caching, RAG) can assume this local stack.